

# LeadOS Admin Portal Screen Requirements Document for Sharad

## 1. Purpose

The Admin Portal is the control center for the enterprise client admin.

For the MVP, Sharad should build the complete Admin Portal UI using static mock data first. Dev will connect real APIs later. The UI should already be structured exactly as if real APIs are integrated.

The Admin Portal should help the client admin understand the full LeadOS pipeline:

Lead captured → AI conversation completed → lead scored as Hot, Warm, or Cold → Hot lead assigned to RM → Warm lead follow up generated → admin dashboard updated

The portal should feel like a real enterprise SaaS dashboard, not a simple hackathon admin page.

## 2. Tech Stack

Use this exact standard stack for the Admin Portal.

- Vite React
- TypeScript
- TanStack Query
- TanStack Router
- Zustand
- React Hook Form plus Zod
- Tailwind CSS
- shadcn/ui
- lucide-react for icons
- Recharts or any clean React chart library for charts

## 3. Development Rule

For now, build everything using static data.

Important rules:

- Do not hardcode data directly inside UI components
- Create mock data files
- Create service functions that return mock API responses
- Use TanStack Query even for mock API calls

- Keep response shapes close to the real API contracts
- Build loading states, empty states, and error states
- Dev will later replace mock services with real API calls

Suggested structure:

```
src/  
  routes/  
  components/  
  features/  
    admin/  
      dashboard/  
      leads/  
      users/  
      analytics/  
      campaigns/  
      followups/  
      knowledge/  
      feedback/  
      settings/  
  services/  
    adminApi.ts  
  mocks/  
    admin.mock.ts  
  store/  
    authStore.ts  
    uiStore.ts  
  types/  
    admin.types.ts
```

## 4. Admin Portal Layout

The portal should have a clean SaaS dashboard layout.

Required layout components:

- Sidebar navigation
- Top header
- Page title area
- Main content area
- Notification icon
- User profile menu
- Client name display
- Logout action

### Sidebar items

Use these sidebar items:

- Dashboard
- Leads
- Conversations
- Analytics
- Campaigns
- Users
- Follow Ups
- Knowledge Base
- Feedback
- Settings

The Users page replaces the earlier RMs page. Users will include both Admins and RMs. Admin can add, remove, deactivate, and view RM performance from here.

## Header

Header should show:

- Current page title
- Client name: FinPartner Pro
- Notification icon
- Admin avatar or initials
- User menu
- Logout option

## 5. Routes

Use TanStack Router.

Required routes:

- /admin/login
- /admin
- /admin/dashboard
- /admin/leads
- /admin/leads/\$leadId
- /admin/conversations
- /admin/analytics
- /admin/campaigns
- /admin/users
- /admin/followups
- /admin/knowledge
- /admin/feedback
- /admin/settings

Default redirect:

/admin → /admin/dashboard

## 6. Shared Types

Use these common types in the UI.

```
type UserRole = "admin" | "rm";

type LeadStatus =
  | "new"
  | "pending_contact"
  | "conversation_completed"
  | "assigned_to_rm"
  | "follow_up_scheduled"
  | "converted"
  | "closed";

type LeadClassification = "hot" | "warm" | "cold";

type ConversationChannel = "chat" | "voice";

type FollowUpStatus = "ready" | "opened" | "sent" | "cancelled";

type UserStatus = "active" | "inactive";
```

## 7. Page 1: Admin Login

### Purpose

Allow admin to login.

During static phase, login should set mock auth state in Zustand and redirect to dashboard.

### Route

/admin/login

### Components

- Login card
- Email input
- Password input
- Submit button
- Demo credentials hint
- Loading state
- Error message

### Validation

Use React Hook Form plus Zod.

Rules:

Email is required  
Email must be valid  
Password is required

## Mock service

loginAdmin(payload)

### Request:

```
{
  "email": "admin@leados.demo",
  "password": "password123"
}
```

### Response:

```
{
  "success": true,
  "data": {
    "accessToken": "mock_access_token",
    "refreshToken": "mock_refresh_token",
    "user": {
      "id": "user_1",
      "clientId": "client_1",
      "name": "Sharad Admin",
      "email": "admin@leados.demo",
      "role": "admin"
    },
    "client": {
      "id": "client_1",
      "name": "FinPartner Pro",
      "slug": "finpartnerpro"
    }
  }
}
```

### Later real API:

POST /auth/login

## 8. Page 2: Dashboard

### Purpose

Dashboard should show whether the AI lead conversion system is working.

### Route

/admin/dashboard

## Sections

- KPI cards
- Lead funnel
- Hot, Warm, Cold distribution
- Language distribution
- Objection trends
- Recent conversations
- Recent Hot leads
- Follow up summary
- RM performance preview

## KPI cards

Show these cards:

Total Leads  
Conversations Completed  
Hot Leads  
Warm Leads  
Cold Leads  
Assigned to RM  
Follow Ups Ready  
Converted

Each card should include:

- Value
- Label
- Icon
- Small trend or helper text if possible

## Charts

Required charts:

- Lead status funnel
- Hot, Warm, Cold chart
- Language distribution chart
- Objection distribution chart

Suggested chart types:

- Funnel as step cards or bar chart
- Classification as donut chart
- Language as bar chart
- Objections as bar chart

## Recent Conversations

Show:

Lead name  
Channel  
Language  
Classification  
Score  
Conversation time  
View lead button

## Recent Hot Leads

Show:

Lead name  
Score  
Assigned RM  
Main objection  
Recommended action

## Mock service

getAdminDashboardOverview()

Later real APIs:

GET /analytics/overview  
GET /analytics/funnel  
GET /analytics/languages  
GET /analytics/objections  
GET /analytics/rmPerformance

Mock response:

```
{
  "success": true,
  "data": {
    "overview": {
      "totalLeads": 50,
      "conversationCompleted": 38,
      "hot": 10,
      "warm": 16,
      "cold": 24,
      "assignedToRm": 10,
      "followUpsScheduled": 16,
      "converted": 5
    },
    "funnel": [
      { "stage": "new", "label": "New", "count": 50 },
      { "stage": "conversation_completed", "label": "Conversation Completed",
```

```

"count": 38 },
  { "stage": "assigned_to_rm", "label": "Assigned to RM", "count": 10 },
  { "stage": "follow_up_scheduled", "label": "Follow Up Scheduled",
"count": 16 },
  { "stage": "converted", "label": "Converted", "count": 5 }
],
"languageStats": [
  { "language": "Hindi", "count": 20 },
  { "language": "English", "count": 12 },
  { "language": "Hinglish", "count": 18 }
],
"objectionStats": [
  { "type": "Existing Broker", "count": 14, "resolved": 10, "unresolved":
4 },
  { "type": "Not Enough Contacts", "count": 9, "resolved": 6,
"unresolved": 3 },
  { "type": "Trust Concern", "count": 6, "resolved": 4, "unresolved": 2 }
]
}
}

```

## 9. Page 3: Leads List

### Purpose

Admin can view, search, filter, sort, and open leads.

### Route

/admin/leads

### Components

- Page header
- Search input
- Filter bar
- Leads table
- Pagination
- Create lead button optional
- Empty state
- Loading skeleton
- Error state

### Table columns

Lead name  
Phone  
City  
Role  
Language



Channel  
Status  
Classification  
Score  
Assigned RM  
Created date  
Actions

## Badges

Use clear badges for:

Hot  
Warm  
Cold  
New  
Pending Contact  
Conversation Completed  
Assigned to RM  
Follow Up Scheduled  
Converted  
Closed

## Filters

Search by name or phone  
Status  
Classification  
Language  
Campaign  
Assigned RM  
Source  
Date range

## Sorting

Created date  
Updated date  
Score  
Name  
Status  
Classification

## Mock service

`getLeads(params)`

Later real API:

`GET /leads`

Query params:

page  
limit  
search  
status  
classification  
language  
campaignId  
assignedRmId  
source  
fromDate  
toDate  
sortBy  
sortOrder

### Mock response:

```
{
  "success": true,
  "data": [
    {
      "id": "lead_1",
      "name": "Amit Sharma",
      "phone": "9876543210",
      "city": "Hyderabad",
      "role": "Financial Advisor",
      "preferredLanguage": "Hinglish",
      "preferredChannel": "chat",
      "status": "assigned_to_rm",
      "classification": "hot",
      "latestScore": 82,
      "latestSummary": "Interested in onboarding and asked for WhatsApp
details.",
      "latestNextAction": "RM should call today before 6 PM.",
      "assignedRm": {
        "id": "user_2",
        "name": "Shivansh Mishra"
      },
      "createdAt": "2026-05-02T10:00:00Z",
      "updatedAt": "2026-05-02T10:20:00Z"
    }
  ],
  "pagination": {
    "page": 1,
    "limit": 20,
    "total": 50,
    "totalPages": 3
  }
}
```

## 10. Page 4: Lead Detail

### Purpose

Admin can inspect a lead completely.

This page proves explainability, transcript storage, objection extraction, score reasoning, and routing.

## Route

/admin/leads/\$leadId

## Sections

- Lead profile card
- Status and classification card
- AI score card
- AI summary
- Objections section
- Transcript section
- RM assignment section
- Follow up section
- Activity timeline optional

## Lead Profile Card

Show:

Name  
Phone  
Email  
City  
Role  
Network size  
Preferred language  
Preferred channel  
WhatsApp consent  
Campaign  
Source  
Created date

## Status Card

Show:

Current status  
Classification  
Latest score  
Assigned RM  
Latest next action

## AI Score Card

Show:

Total score  
Classification  
Readiness score  
Engagement score  
Fit score  
Score reason  
Positive signals  
Negative signals  
Recommended next action

### Suggested UI:

- Large score number
- Classification badge
- Score breakdown cards
- Positive and negative signals as chips

## AI Summary

### Show:

What was discussed  
What the lead cared about  
Lead intent  
Next recommended action

## Objections Section

### Each objection card should show:

Objection type  
Lead statement  
AI response summary  
Resolution status

### Resolution status values:

Resolved  
Partially Resolved  
Unresolved

## Transcript Section

### Show chat style transcript.

### Each message should show:

Role  
Message content

Timestamp  
Language optional

## RM Assignment Section

Show:

Assigned RM  
RM task status  
Suggested RM opening line  
Task priority  
Due time optional

Admin actions optional:

Assign RM  
Reassign RM

## Follow Up Section

Show for Warm leads.

Channel  
Status  
Message preview  
WhatsApp link  
Open WhatsApp button  
Mark Sent button

## Mock service

`getLeadDetail(leadId)`

Later real API:

`GET /leads/:leadId`

Mock response:

```
{
  "success": true,
  "data": {
    "lead": {
      "id": "lead_1",
      "name": "Amit Sharma",
      "phone": "9876543210",
      "email": "amit@example.com",
      "city": "Hyderabad",
      "role": "Financial Advisor",
      "networkSize": "25 to 30",
      "preferredLanguage": "Hinglish",
    }
  }
}
```

```
    "preferredChannel": "chat",
    "whatsappConsent": true,
    "status": "assigned_to_rm",
    "classification": "hot",
    "latestScore": 82,
    "latestSummary": "Lead asked for onboarding details and responded
positively to daily payouts.",
    "latestNextAction": "RM should call today before 6 PM."
  },
  "campaign": {
    "id": "campaign_1",
    "name": "Demo Partner Campaign"
  },
  "assignedRm": {
    "id": "user_2",
    "name": "Shivansh Mishra"
  },
  "latestConversation": {
    "id": "conv_1",
    "channel": "chat",
    "status": "completed",
    "language": "hinglish",
    "durationSeconds": 420,
    "summary": "Discussed brokerage share, daily payouts, and onboarding."
  },
  "messages": [
    {
      "id": "msg_1",
      "role": "user",
      "content": "I already have another broker.",
      "sequenceNo": 1,
      "timestamp": "2026-05-02T10:05:00Z"
    },
    {
      "id": "msg_2",
      "role": "assistant",
      "content": "That is actually a good sign because you already
understand this business. Are you getting 100 percent brokerage and daily
payouts?",
      "sequenceNo": 2,
      "timestamp": "2026-05-02T10:05:10Z"
    }
  ],
  "score": {
    "id": "score_1",
    "classification": "hot",
    "totalScore": 82,
    "readinessScore": 34,
    "engagementScore": 29,
    "fitScore": 19,
    "reason": "Asked for onboarding details, has relevant network, and
existing broker objection was resolved.",
    "recommendedNextAction": "RM should call today before 6 PM.",
    "positiveSignals": ["Asked for onboarding", "Has investor network",
"WhatsApp consent given"],
    "negativeSignals": [],
    "objections": [
```

```

        {
          "type": "Existing Broker",
          "status": "resolved",
          "leadStatement": "I already have another broker.",
          "aiResponseSummary": "Compared 100 percent brokerage and daily
payouts."
        }
      ]
    },
    "rmTask": {
      "id": "task_1",
      "status": "pending",
      "priority": "high",
      "recommendedAction": "Call today before 6 PM.",
      "suggestedOpeningLine": "Hi Amit, I saw you spoke with our AI assistant
about brokerage share and daily payouts."
    },
    "followUp": null
  }
}

```

## 11. Page 5: Conversations

### Purpose

Admin can see all AI conversations across leads.

### Route

/admin/conversations

### Components

- Conversation list
- Filters
- Conversation detail drawer or page
- Transcript preview
- Channel badge
- Language badge
- Classification badge

### Columns

Lead name  
Channel  
Language  
Status  
Duration  
Classification  
Score

Started time  
Action

## Filters

Channel  
Language  
Status  
Classification  
Date range

## Mock service

`getConversations(params)`

Later API options:

`GET /conversations`

If Dev does not build a dedicated conversations list API, this page can use mock data now and later derive from leads plus lead detail APIs.

# 12. Page 6: Analytics

## Purpose

Detailed analytics beyond the dashboard.

## Route

`/admin/analytics`

## Sections

- Lead funnel
- Hot, Warm, Cold distribution
- Language distribution
- Objection distribution
- RM performance
- Campaign performance
- Follow up performance

## Recommended charts

Funnel or step chart  
Donut chart for classification  
Bar chart for languages



Bar chart for objections  
RM performance table  
Campaign performance table

## Mock services

```
getAnalyticsOverview()  
getLeadFunnel()  
getLanguageStats()  
getObjectionStats()  
getRmPerformance()
```

### Later real APIs:

```
GET /analytics/overview  
GET /analytics/funnel  
GET /analytics/languages  
GET /analytics/objections  
GET /analytics/rmPerformance
```

## 13. Page 7: Campaigns

### Purpose

Admin can view lead performance by campaign.

For MVP, one default campaign is enough, but UI should support multiple campaigns.

### Route

/admin/campaigns

### Components

- Campaign table or cards
- Campaign metrics
- Create campaign button optional
- Campaign detail drawer optional

### Columns

Campaign name  
Source  
Total leads  
Hot leads  
Warm leads  
Cold leads  
Converted leads

Active status  
Actions

## Mock service

getCampaigns(params)  
createCampaign(payload)

### Later APIs:

GET /campaigns  
POST /campaigns

### Mock response:

```
{
  "success": true,
  "data": [
    {
      "id": "campaign_1",
      "name": "Demo Partner Campaign",
      "source": "website",
      "isActive": true,
      "leadCount": 50,
      "hotCount": 10,
      "warmCount": 16,
      "coldCount": 24,
      "convertedCount": 5
    }
  ],
  "pagination": {
    "page": 1,
    "limit": 20,
    "total": 1,
    "totalPages": 1
  }
}
```

## 14. Page 8: Users

### Purpose

Admin can manage all users under the client workspace.

This includes Admin users and RM users. For the MVP, this page is important because Hot leads are assigned to RMs, so the admin needs to view and manage RM accounts.

### Route

/admin/users

## Access

Admin only

## Main features

- View all users
- Search users by name or email
- Filter by role
- Filter by active status
- View assigned lead count for each RM
- View pending task count
- View converted count
- Add new RM
- Remove or deactivate RM
- Open user detail drawer
- Reactivate user optional

## Recommended tabs

All Users  
Admins  
RMs  
Inactive

## Table columns

Name  
Email  
Role  
Active Status  
Assigned Leads  
Pending Tasks  
Converted Leads  
Created Date  
Actions

## Filters

Search  
Role  
Active or inactive  
Sort by name  
Sort by created date  
Sort by assigned leads  
Sort by converted leads

## Actions

### Required actions:

View details  
Add RM  
Deactivate user

### Optional actions:

Reactivate user  
Edit name  
Edit role

Do not permanently delete users in UI. Use deactivate instead. The button can say Remove RM, but internally it should mean deactivate.

## Add RM Modal

Use React Hook Form plus Zod.

### Fields:

Name  
Email  
Temporary password optional

For static phase, Add RM should add to local mock state or show success toast.

### Validation:

Name required  
Email required  
Valid email required

### Mock request:

```
{
  "name": "New RM",
  "email": "newrm@leados.demo"
}
```

### Mock response:

```
{
  "success": true,
  "data": {
    "rm": {
      "id": "user_4",
      "name": "New RM",
      "email": "newrm@leados.demo",
      "role": "rm",
      "isActive": true
    }
  }
}
```

```
    }  
  }  
}
```

Later real API:

```
POST /users/rms
```

## Deactivate RM Modal

When admin clicks Remove or Deactivate:

Show confirmation dialog.

Dialog text:

This will deactivate the RM account. Existing assigned leads will remain visible to admins. You can reassign leads later.

Mock request:

```
{  
  "isActive": false  
}
```

Later real API:

```
PATCH /users/:userId
```

Response:

```
{  
  "success": true,  
  "data": {  
    "user": {  
      "id": "user_2",  
      "isActive": false  
    }  
  }  
}
```

## User detail drawer

Show:

Name  
Email  
Role  
Status  
Assigned leads

Pending tasks  
Converted leads  
Recent assigned leads  
Recent activity

## Mock service

getUsers(params)  
createRM(payload)  
updateUser(userId, payload)  
deactivateUser(userId)

## Later APIs:

GET /users  
GET /users/rms  
POST /users/rms  
PATCH /users/:userId

## Mock response:

```
{
  "success": true,
  "data": [
    {
      "id": "user_1",
      "name": "Sharad Admin",
      "email": "admin@leados.demo",
      "role": "admin",
      "isActive": true,
      "assignedLeadCount": 0,
      "pendingTaskCount": 0,
      "convertedCount": 0,
      "createdAt": "2026-05-02T09:00:00Z"
    },
    {
      "id": "user_2",
      "name": "Shivansh Mishra",
      "email": "rm@leados.demo",
      "role": "rm",
      "isActive": true,
      "assignedLeadCount": 10,
      "pendingTaskCount": 3,
      "convertedCount": 5,
      "createdAt": "2026-05-02T09:10:00Z"
    }
  ],
  "pagination": {
    "page": 1,
    "limit": 20,
    "total": 2,
    "totalPages": 1
  }
}
```

## 15. Page 9: Follow Ups

### Purpose

Admin can see Warm lead follow ups.

### Route

`/admin/followups`

### Components

- Follow up table
- Status filters
- Message preview
- Open WhatsApp button
- Mark Sent button

### Columns

Lead name  
Phone  
Classification  
Channel  
Status  
Message preview  
Created time  
Actions

### Actions

Open WhatsApp  
Mark Sent  
Cancel optional

### Mock services

`getFollowUps(params)`  
`openFollowUp(followUpId)`  
`markFollowUpSent(followUpId)`

### Later APIs:

`GET /followups`  
`POST /followups/:followUpId/open`  
`POST /followups/:followUpId/markSent`

Mock response:

```
{
  "success": true,
  "data": [
    {
      "id": "follow_1",
      "lead": {
        "id": "lead_2",
        "name": "Priya Mehta",
        "phone": "9876543211",
        "classification": "warm"
      },
      "channel": "whatsapp",
      "status": "ready",
      "message": "Hi Priya, thanks for speaking with us. Here is the
onboarding link...",
      "waMeLink": "https://wa.me/919876543211?text=Hello",
      "createdAt": "2026-05-02T10:30:00Z"
    }
  ]
}
```

## 16. Page 10: Knowledge Base

### Purpose

Admin can see documents used by AI.

For MVP, documents can be static seeded docs.

### Route

/admin/knowledge

### Components

- Document list
- Document type filter
- Document detail view
- Active status badge
- Create document button optional

### Document types

Script  
FAQ  
Onboarding  
Objection Guide

### Mock services



```
getKnowledgeDocuments (params)
getKnowledgeDocument (documentId)
createKnowledgeDocument (payload)
```

Later APIs:

```
GET /knowledge
GET /knowledge/:documentId
POST /knowledge
```

## 17. Page 11: Feedback

### Purpose

Admin can review RM feedback on AI scoring.

### Route

/admin/feedback

### Components

- Feedback table
- Filters
- Lead link
- RM name
- Original vs corrected classification
- Comment

### Columns

```
Lead name
RM name
Feedback type
Original classification
Corrected classification
Original score
Corrected score
Comment
Created date
```

### Mock service

```
getFeedback (params)
```

Later API:

```
GET /feedback
```

Mock response:

```
{
  "success": true,
  "data": [
    {
      "id": "feedback_1",
      "leadId": "lead_1",
      "leadName": "Amit Sharma",
      "rmId": "user_2",
      "rmName": "Shivansh Mishra",
      "originalClassification": "hot",
      "correctedClassification": "warm",
      "originalScore": 82,
      "correctedScore": 65,
      "feedbackType": "score_wrong",
      "comment": "Lead was interested but not ready for onboarding.",
      "createdAt": "2026-05-02T11:00:00Z"
    }
  ]
}
```

## 18. Page 12: Settings

### Purpose

Admin can see system configuration.

For MVP, most settings can be display only.

### Route

/admin/settings

### Sections

Client information  
Supported languages  
Scoring thresholds  
WhatsApp message template  
Voice provider  
LLM provider  
RAG status  
Memory status  
Automation status

### Values to show

Supported languages:

English  
Hindi  
Hinglish

### Scoring thresholds:

Hot: 70 plus  
Warm: 40 to 69  
Cold: below 40

### Providers:

Voice: Vapi  
LLM: OpenAI  
RAG: LangChain  
Memory: Mem0  
Automation: n8n  
Database: Supabase

## 19. Optional Admin Data Chatbot

### Purpose

Differentiator for the admin portal.

It can appear on dashboard or as a floating drawer.

### Supported static prompts

Show Hot Hindi leads  
Which objection is most common?  
Show Warm leads needing follow up  
Show campaign performance  
Show RM performance

### UI

- Floating chat button
- Chat drawer
- Input field
- Suggested prompt chips
- Response card with lead links

### Later API:

POST /dataChat/query

Not required in first build.

## 20. Shared Components

Build reusable components.

### Layout components

AdminLayout  
Sidebar  
Header  
PageHeader  
Breadcrumbs optional

### Data components

StatCard  
StatusBadge  
ClassificationBadge  
LanguageBadge  
ScoreBadge  
EmptyState  
LoadingSkeleton  
ErrorState  
DataTable  
FilterBar  
Pagination

### Lead components

LeadCard  
LeadProfileCard  
ScoreCard  
ScoreBreakdown  
ObjectionCard  
TranscriptViewer  
RMTaskCard  
FollowUpCard

### Analytics components

FunnelChart  
ClassificationChart  
LanguageChart  
ObjectionChart  
RMPerformanceTable

### User components

UserTable  
UserRoleBadge  
UserStatusBadge  
AddRMDialog

DeactivateUserDialog  
UserDetailDrawer

## Form components

SearchInput  
FilterSelect  
DateRangeFilter  
CreateCampaignForm optional  
AssignRMForm optional  
AddRMForm

## 21. Mock Data Requirements

Create static mock data for:

Admin user  
Client  
Campaigns  
Users  
RMs  
Leads  
Conversations  
Messages  
Scores  
Follow ups  
Notifications  
Knowledge documents  
Feedback  
Analytics

Minimum mock leads:

One Hot lead with RM task  
One Warm lead with WhatsApp follow up  
One Cold lead closed  
One Pending Contact lead  
One Conversation Completed lead

Minimum mock users:

One Admin  
Two active RMs  
One inactive RM

## 22. Zustand Store

Use Zustand only for local app state.

Store:

Auth user  
Client  
Sidebar collapsed state  
Theme optional  
Global selected filters optional

Do not store server data in Zustand. Server data should come through TanStack Query.

## 23. TanStack Query Hooks

Create hooks for mock service functions.

Examples:

```
useAdminDashboard()  
useLeads(params)  
useLeadDetail(leadId)  
useAnalytics()  
useCampaigns(params)  
useUsers(params)  
useFollowUps(params)  
useKnowledgeDocuments(params)  
useFeedback(params)
```

Mock functions should return promises and use the same response structure as real APIs.

## 24. Loading, Empty, and Error States

Every main page should have:

### Loading state

Use skeletons.

### Empty state

Examples:

No leads found  
No users found  
No follow ups ready  
No feedback submitted  
No knowledge documents available

### Error state

Show:

Friendly message  
Retry button

## 25. Visual Design Direction

Admin Portal should feel:

Clean  
Modern  
Enterprise SaaS  
Light theme  
Data rich but not cluttered  
Clear badges  
Easy to scan

Use shadcn/ui:

Card  
Button  
Badge  
Table  
Tabs  
Dialog  
Sheet  
Dropdown Menu  
Select  
Input  
Textarea  
Skeleton  
Separator  
Tooltip  
Alert

## 26. Most Important Demo Screens

If time is limited, prioritize these:

Dashboard  
Leads List  
Lead Detail  
Users  
Analytics  
Follow Ups

These screens prove the admin side of the product.

## 27. Admin Portal Build Priority

**Priority 1**

Admin layout  
Login  
Dashboard  
Leads list  
Lead detail  
Users page  
Analytics cards and charts

## Priority 2

Follow Ups page  
Campaigns page  
Knowledge Base page  
User add and deactivate flow

## Priority 3

Feedback page  
Settings page  
Data chatbot  
Extra polish and animations

# 28. Final Success Criteria

Sharad's Admin Portal is complete for MVP if:

- Admin can login using mock auth
- Dashboard shows funnel and key metrics
- Lead list shows leads with filters and sorting UI
- Lead detail shows profile, score, transcript, objections, RM task, and follow up
- Users page shows admins and RMs
- Admin can add a new RM in mock UI
- Admin can deactivate or remove RM in mock UI
- Analytics page shows language, classification, objection, and funnel insights
- Follow Ups page shows Warm lead WhatsApp links
- UI uses static data but is ready for API integration
- All service functions return data in expected API response format